# Git and GitHub Course

For beginners. Simple steps, examples, common errors and fixes. Built for the GitHub profile: fussy0064.

## Part 0: The Big Picture

- Git = a tool on your computer. It saves versions of your project.
- GitHub = a website that stores your Git projects online. You share and team up there.
- Git works offline. GitHub needs internet.

### The 4 areas of Git

| Area | What it is |
|---|---|
| Working directory | Your project folder. You edit files here. |
| Staging area | A waiting room. You pick what goes into the next save. |
| Local repository | The .git folder. Your saved commits live here. |
| Remote repository | The copy on GitHub. |

```
Working dir  --git add-->  Staging  --git commit-->  Local repo  --git push-->  GitHub
```

### Key words

- Repository (repo): a project folder tracked by Git.
- Commit: one saved snapshot with a message.
- Branch: a separate line of work.
- Merge: join one branch into another.
- Clone: download a full copy of a repo.
- Push: send commits to GitHub. Pull: get commits from GitHub.
- HEAD: a pointer to where you are now.

### Install and first setup

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --list
```

**Output:** git version 2.43.0

**Note:** Old guides use master as the main branch. GitHub now uses main. This course uses main.

**Error:** Author identity unknown. Please tell me who you are.

**Fix:** Run the two git config lines above (user.name and user.email).

## Part 1: Basic Commands

### 1. git init - start a new repo

Use it when you start a new project. It creates the hidden .git folder.

```
mkdir my-project
cd my-project
git init
```

**Output:** Initialized empty Git repository in /home/user/my-project/.git/

**Error:** fatal: not a git repository (or any of the parent directories): .git

**Fix:** You are in the wrong folder. Use cd to go to your project, or run git init.

## 2. git clone - copy an existing repo

Use it to download a repo from GitHub, with all files and history.

```
git clone https://github.com/fussy0064/fussy.git
cd fussy
```

**Output:** Cloning into 'fussy'... done.

**Error:** fatal: repository '...' not found

**Fix:** Check the URL spelling. If the repo is private, log in with a token or SSH key.

**Error:** fatal: destination path 'fussy' already exists and is not an empty directory.

**Fix:** Delete or rename that folder, or clone into a new name: git clone URL new-name

## 3. git status - see what is going on

Your most used command. Run it often.

```
echo "Hello Git" > hello.txt
git status
```

```
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.txt
```

| Message | Meaning |
|---|---|
| Untracked files | Git does not know this file yet. |
| Changes not staged for commit | You edited a tracked file but did not git add it. |
| Changes to be committed | Staged. Ready for git commit. |
| nothing to commit, working tree clean | No changes. All saved. |

## 4. git add - move changes to staging

```
git add hello.txt     # one file
git add .             # everything in this folder
git add -A            # everything in the whole repo
```

**Error:** fatal: pathspec 'helo.txt' did not match any files

**Fix:** Typo in the file name. Run ls to see real names.

**Security tip:** Before git add ., check git status. Never add passwords, .env files or tokens.

## 5. git commit - save a snapshot

```
git commit -m "Add hello file"
```

```
[main (root-commit) a1b2c3d] Add hello file
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
```

- Write short, clear messages. Example: "Fix login bug".
- git commit -am "msg" stages and commits tracked files in one step. It skips new files.

**Error:** nothing to commit, working tree clean

**Fix:** You changed nothing, or forgot to save the file. Edit, then git add again.

**Error:** Author identity unknown

**Fix:** Set user.name and user.email (see Part 0).

**Typo in last message?** git commit --amend -m "New message" (only before you push).

## 6. git remote - connect to GitHub

Needed only for repos you made with git init. Cloned repos already have it.

```
git remote add origin https://github.com/fussy0064/fussy.git
git remote -v
```

```
origin  https://github.com/fussy0064/fussy.git (fetch)
origin  https://github.com/fussy0064/fussy.git (push)
```

**Error:** error: remote origin already exists.

**Fix:** git remote set-url origin NEW_URL

## 7. git push - send commits to GitHub

```
git branch -M main
git push -u origin main
```

```
To https://github.com/fussy0064/fussy.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

The -u flag links your branch to GitHub. Next time, just use git push.

**Error:** error: src refspec main does not match any

**Fix:** You have no commit yet, or your branch is named master. Commit first, then run git branch -M main.

**Error:** ! [rejected] main -> main (fetch first) / Updates were rejected because the remote contains work that you do not have locally

**Fix:** git pull --rebase origin main, then git push.

**Error:** remote: Support for password authentication was removed. fatal: Authentication failed

**Fix:** GitHub no longer takes your password. Use a Personal Access Token as the password, or use SSH (Part 4).

**Error:** remote: error: File ... is 120 MB; this exceeds GitHub's file size limit of 100 MB

**Fix:** Remove the big file from the commit, add it to .gitignore, or use Git LFS.

## 8. git pull - get updates from GitHub

git pull = git fetch + git merge. Always pull before you push when working in a team.

```
git pull origin main
```

```
Updating a1b2c3d..d4e5f6a
Fast-forward
 README.md | 2 ++
```

**Error:** fatal: refusing to merge unrelated histories

**Fix:** git pull origin main --allow-unrelated-histories

**Error:** error: Your local changes would be overwritten by merge

**Fix:** Commit your changes first, or use git stash (Part 3).

**Error:** CONFLICT (content): Merge conflict in file.txt

**Fix:** See "Merge conflicts" in Part 2.

### 9. Extra daily commands

```
git log --oneline        # short history
git diff                # see unstaged changes
git diff --staged       # see staged changes
git rm file.txt         # delete a tracked file
git rm --cached file    # untrack, keep the file on disk
```

```
a1b2c3d (HEAD -> main, origin/main) Add hello file
```

**.gitignore - files Git must skip**

```
echo ".env" >> .gitignore
echo "*.log" >> .gitignore
echo "node_modules/" >> .gitignore
```

**Problem:** You added a file to .gitignore but Git still tracks it.

**Fix:** git rm --cached filename, then commit. .gitignore only affects untracked files.

## Part 2: Branches

A branch is a safe copy of your work line. You try new things there. The main branch stays clean.

### 1. git branch - list, create, delete

```
git branch                 # list branches
git branch feature-login   # create (stay where you are)
git branch -d feature-login   # delete if merged
git branch -D feature-login   # force delete
```

```
* main
  feature-login
```

The star shows your current branch.

**Error:** fatal: A branch named 'feature-login' already exists.

**Fix:** Pick a new name, or switch to it.

**Error:** error: The branch 'x' is not fully merged.

**Fix:** Merge it first. If you really want to lose it, use -D.

### 2. git checkout - switch branches

```
git checkout feature-login      # switch
git checkout -b feature-login    # create and switch
```

```
Switched to a new branch 'feature-login'
```

Newer Git also has git switch and git switch -c. They do the same job and are easier to read.

**Error:** error: pathspec 'feature' did not match any file(s) known to git

**Fix:** Branch name is wrong. Run git branch to see names. Use -b to create it.

**Error:** error: Your local changes to the following files would be overwritten by checkout

**Fix:** Commit your work or run git stash, then switch.

**Detached HEAD:** You checked out a commit, not a branch. Go back with git switch main. To keep the work: git switch -c new-branch.

### 3. git merge - join branches

Go to the branch you want to receive the changes. Then merge.

```
git checkout feature-login
echo "login page" > login.txt
```

```
git add login.txt
git commit -m "Add login page"

git checkout main
git merge feature-login
```

```
Updating a1b2c3d..d4e5f6a
Fast-forward
 login.txt | 1 +
```

- Fast-forward: main had no new commits, so Git just moves the pointer.
- Merge commit: both branches changed, so Git makes a new commit to join them.

### Merge conflicts

A conflict happens when two branches change the same line.

```
Auto-merging file.txt
CONFLICT (content): Merge conflict in file.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Open the file. You will see markers:

```
<<<<<<< HEAD
Title from main
=======
Title from feature
>>>>>>> feature-login
```

Steps to fix:

- Keep the text you want. Delete the three marker lines.
- git add file.txt
- git commit (no message needed, Git fills one)
- Changed your mind? git merge --abort

## 4. git rebase - replay your commits on top

Rebase moves your branch commits to the tip of another branch. History looks like a straight line.

```
git checkout feature-login
git rebase main
```

```
Successfully rebased and updated refs/heads/feature-login.
```

| | **merge** | **rebase** |
|---|---|---|
| History | Keeps real history, adds a merge commit | Clean straight line |
| Safe on shared branches? | Yes | No. It rewrites commits. |

**Golden rule:** Never rebase a branch other people already use.

**Conflict during rebase:** Fix the file, then git add file, then git rebase --continue. To stop: git rebase --abort.

**Error after rebase and push:** ! [rejected] (non-fast-forward)

**Fix:** Only on your own branch: git push --force-with-lease. It is safer than --force.

## 5. Push a branch to GitHub

```
git push -u origin feature-login
git push origin --delete feature-login   # delete on GitHub
```

# Part 3: Advanced Commands

## 1. git stash - park unfinished work

You are in the middle of work and must switch branches. Stash saves it for later.

```
git stash push -m "WIP: login form"
git stash list
git stash pop       # restore and remove from list
git stash apply     # restore and keep in list
git stash drop      # delete newest stash
git stash clear     # delete all stashes
```

```
Saved working directory and index state On main: WIP: login form
stash@{0}: On main: WIP: login form
```

- New untracked files are not stashed by default. Use git stash -u.

**Error:** No local changes to save

**Fix:** Nothing changed. Nothing to stash.

**Conflict on pop:** Fix like a merge conflict. The stash stays in the list until you drop it.

## 2. git cherry-pick - copy one commit

Take a single commit from another branch and put it on your current branch.

```
git log --oneline feature-login
# d4e5f6a Fix typo in header

git checkout main
git cherry-pick d4e5f6a
```

```
[main 9f8e7d6] Fix typo in header
 1 file changed, 1 insertion(+)
```

**Error:** CONFLICT after cherry-pick

**Fix:** Edit file, git add, then git cherry-pick --continue. Or stop with git cherry-pick --abort.

**Error:** fatal: bad revision 'd4e5f6a'

**Fix:** Wrong hash. Copy it again from git log --oneline.

## 3. git revert - undo safely

Revert makes a new commit that cancels an old one. History is kept. Safe for pushed commits.

```
git revert HEAD              # undo last commit
git revert d4e5f6a --no-edit # undo a chosen commit
```

```
[main 7c6b5a4] Revert "Add login page"
 1 file changed, 1 deletion(-)
```

**Error:** error: commit ... is a merge but no -m option was given

**Fix:** git revert -m 1 MERGE_COMMIT_HASH. The number 1 means keep the main side.

## 4. git reset - move back in time

Reset moves your branch pointer back. It can erase work. Use it on local commits only.

| Mode | What happens | Your files |
|---|---|---|
| `git reset --soft HEAD~1` | Undo commit | Changes stay staged |
| `git reset --mixed HEAD~1` | Undo commit and unstage (default) | Changes stay in files |

| Mode | What happens | Your files |
|---|---|---|
| `git reset --hard HEAD~1` | Undo commit and wipe changes | Changes are deleted |

```
git reset --soft HEAD~1       # fix the last commit message or content
git reset HEAD file.txt        # unstage one file
git reset --hard HEAD~1       # DANGER: lose changes
```

HEAD~1 means one commit before now.

**Oops, reset --hard by mistake?** Run git reflog. Find the old hash. Then git reset --hard HASH.

```
git reflog
# a1b2c3d HEAD@{1}: commit: Add hello file
```

**revert vs reset:** Pushed already? Use revert. Not pushed? reset is fine.

### 5. Other useful tools

```
git commit --amend            # change last commit
git restore file.txt           # throw away unstaged edits
git restore --staged file.txt  # unstage
git tag v1.0                   # label a release
git push origin v1.0
```

# Part 4: GitHub Setup and Teamwork

## SSH keys (no password each push)

```
ssh-keygen -t ed25519 -C "you@example.com"
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

- Copy the output of the last command. It is your public key.
- GitHub > Settings > SSH and GPG keys > New SSH key > paste > save.

```
ssh -T git@github.com
```

**Output:** Hi fussy0064! You've successfully authenticated.

**Important:** Never cat or share the file without .pub. That is your private key. Some old guides show cat ~/.ssh/id_rsa. Do not copy that.

**Error:** Permission denied (publickey)

**Fix:** Key not added to GitHub, or ssh-agent is not running. Redo the steps above.

**Switch an HTTPS repo to SSH:** git remote set-url origin git@github.com:fussy0064/fussy.git

## Personal Access Token (HTTPS option)

- GitHub > Settings > Developer settings > Personal access tokens.
- Give the smallest permission needed and a short expiry.
- Use the token as the password when Git asks.
- Never put a token in code, in a commit, or in a chat.

## Fork and Pull Request (open source)

- Fork the repo on GitHub.
- git clone your fork, then git checkout -b my-fix.

- Make changes, commit, git push -u origin my-fix.
- On GitHub click Compare and pull request. Explain your change.
- Owner reviews and merges.

### Security rules

- Turn on two-factor authentication (2FA).
- Keep secrets in .env and list .env in .gitignore.
- Pushed a secret by mistake? Revoke it at once and make a new one. Deleting the commit is not enough.
- Use branch protection and pull requests on important repos.

## Part 5: Build Your Git Profile

Your repo: https://github.com/fussy0064/fussy.git

I cannot open or push to your GitHub. Run these steps on your computer.

### Step 1: Clone and set up

```
git clone https://github.com/fussy0064/fussy.git
cd fussy
mkdir notes
```

### Step 2: Add your work

- Put this course file inside the notes folder.
- Create a README.md that says who you are and what the repo holds.

```
# Fussy - Learning Git
My Git and GitHub course notes and practice.

## Contents
- notes/ : course notes
- practice/ : daily exercises
```

### Step 3: First commit and push

```
git status
git add .
git commit -m "Add course notes and README"
git push origin main
```

### Step 4: Build the habit with branches

```
git checkout -b practice-branches
# do the exercise, save files
git add .
git commit -m "Practice: branches"
git push -u origin practice-branches
# open a Pull Request on GitHub, then merge it
```

### Step 5: Make your profile page

- Create a new public repo named exactly fussy0064 (same as your username).
- Add a README.md. GitHub shows it at the top of your profile.
- Write: short intro, skills, what you are learning, links.
- Pin your best 3 to 6 repos.
- Add a photo, a short bio and your location.

```
# Hi, I am Fussy
```

```
Student. Learning programming and cyber security.

## Skills
Git, GitHub, (add yours)

## Projects
- fussy : Git course notes and practice
```

### Step 6: Weekly plan

| Day | Task |
|---|---|
| Day 1 | init, add, commit, push your first repo. |
| Day 2 | Branch, merge, fix one conflict on purpose. |
| Day 3 | Stash and cherry-pick practice. |
| Day 4 | Revert and reset on a test repo. Use reflog to recover. |
| Day 5 | Fork a friend's repo and send a pull request. |
| Day 6 | Polish README and profile. Pin repos. |
| Day 7 | Review errors. Write what you learned. |

## Part 6: Error Quick Table

| Error | Cause | Fix |
|---|---|---|
| fatal: not a git repository | Wrong folder | cd to project or git init |
| Author identity unknown | Name and email not set | `git config --global user.name and user.email` |
| pathspec did not match any files | File name typo | Check with ls |
| src refspec main does not match any | No commits, or branch is master | Commit first, git branch -M main |
| rejected: fetch first | Remote has newer commits | `git pull --rebase origin main` |
| Authentication failed | Passwords are not accepted | Use token or SSH |
| Permission denied (publickey) | SSH key missing on GitHub | Add .pub key to GitHub |
| remote origin already exists | Remote already set | `git remote set-url origin URL` |
| unrelated histories | Two repos started separately | --allow-unrelated-histories |
| CONFLICT (content) | Same line changed twice | Edit, git add, git commit |
| local changes would be overwritten | Unsaved work | Commit or git stash |
| branch not fully merged | Unmerged work | Merge first, or git branch -D |
| Detached HEAD | Checked out a commit | `git switch main` |
| File exceeds 100 MB | Big file in commit | Remove it, .gitignore, or Git LFS |

## Part 7: Cheat Sheet

```
git init                      start repo
git clone URL                 download repo
git status                    see state
git add .                     stage all
git commit -m "msg"           save snapshot
git push / git pull           send / get
git branch / checkout -b      list / create branch
git merge NAME                join branch
git rebase main               replay on main
git stash / stash pop         park / restore work
git cherry-pick HASH          copy one commit
git revert HASH               safe undo
git reset --soft HEAD~1       undo commit, keep work
git reflog                    find lost commits
git log --oneline             short history
```

### Reflection questions

- Which command did you use most?
- Which error took longest to fix, and how did you fix it?
- What will you practice next week?